PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
100-ft Ring Road, Bengaluru – 560 085, Karnataka, India

Capstone Project Report (Phase-1) on

Medicine Dispenser

Submitted by

SUMUKHA ML - (PES1PG24CA184)

Aug 2025 – Jan 2026

under the guidance of

Mr. Santosh Katti

Assistant Professor
Department of Computer Applications,
PESU, Bengaluru – 560085

FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER APPLICATIONS
PROGRAM – MASTER OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that the project entitled

Medicine Dispenser

is a bonafide work carried out by

Sumukha ML-PES1PG24CA184

in partial fulfilment for the completion of Capstone Project, Phase-1 work in the Program of Study MCA under rules and regulations of PES University, Bengaluru during the period Aug. 2025 – Jan 2026. The project report has been approved as it satisfies the academic requirements of 3rd semester MCA.

Signature with date
Guide
Mr. Santosh Katti, Assistant Professor,
Department of Computer Applications,
PES University, Bengaluru - 560085

Signature with date & Seal
Chairperson
Dr. Veena S
Department of Computer Applications, PES University, Bengaluru - 560085

DECLARATION

I, **Sumukha ML,** bearing **PES1PG24CA184** hereby declare that the Capstone project phase-1 entitled, ***Medicine Dispenser,*** is an original work done by me under the guidance of **Mr.Santosh Katti,** Assistant Professor, PES University, and is being submitted in partial fulfillment of the requirements for completion of 3rd Semester course in the Program of Study **MCA**. All corrections/suggestions indicated for internal assessment have been incorporated in the report.

PLACE: Bengaluru

DATE:

NAME AND SIGNATURE OF THE CANDIDATE

ABSTRACT

Medication adherence remains a significant challenge in healthcare, especially for elderly patients and individuals managing chronic illnesses. The traditional reminder-based systems heavily rely on user compliance and cannot guarantee the correctness of medicine dispensing. This paper proposes an *Intelligent Voice-Driven Smart Medicine Dispenser* that combines *Natural Language Processing (NLP)*, web technologies, and the *Internet of Things (IoT)* for automatic and accurate medication management.

The architecture of the system includes a web application based on React, an NLP engine powered by spaCy, a database on the cloud, and a dispensing unit controlled by Raspberry Pi. Patients interact with the system by recording voice commands through the provided web interface. The recorded audio is converted to text and then processed with NLP to identify medicine names, dosage intent, and temporal references. The system intelligently maps spoken inputs against the patient's prescribed medicine list, allowing for robust command recognition that tolerates similar or approximate medicine names.

Caregivers can configure multiple patients with medicine, dosage, and schedule through the web application and enable reminders and alerts. Upon a command like “dispense today’s medicine” by the patient, the system verifies it and retrieves all medicines scheduled for that particular patient. The Raspberry Pi listens continuously for any dispensing instruction from the cloud and runs DC motors and servo motors to dispense the correct medicines at the right time. Visual feedback is given through an LCD display and LEDs, while audio gives step-by-step guidance to the patient during the dispensing process.

The proposed system enhances medication adherence, decreases human error, and significantly improves accessibility for elderly and dependent users. The proposed approach is both highly scalable and intelligent, integrating voice interaction, NLP-driven intent recognition, and IoT-based automation into modern healthcare management.

Table of Content

CHAPTER 1
INTRODUCTION

1.1 Problem Scenario

Many patients, especially older adults, struggle to remember complex medication schedules or differentiate between similar pills. Caregivers often manage multiple patients manually, which increases the chance of error. Existing reminder systems can notify users but cannot confirm whether the right medicine was taken at the right time. Hence, the lack of automation and real-time verification leads to inconsistent adherence and potential health risks.

1.2 Proposed Solution

The proposed Intelligent Voice-Driven Smart Medicine Dispenser addresses these challenges using an integrated system that combines AI-based NLP, web interfaces, and IoT hardware.

The system consists of:

- A React-based web application for user and caregiver interaction.
- A spaCy-powered NLP engine that processes voice commands to detect medicine names, dosage, and timing.
- A cloud database for storing prescriptions, schedules, and patient data.
- A Raspberry Pi–based dispensing unit with DC and servo motors to deliver medicines accurately.

Patients interact through voice commands like *"dispense my evening medicine"* or *"show today's schedule."* The command is processed using NLP, matched with the prescription data, and verified before dispensing. The Raspberry Pi controls the motors to release the correct medicines, while LEDs, an LCD screen, and audio alerts provide feedback.

Caregivers can use the web dashboard to manage multiple patients, set reminders, and monitor adherence remotely. The system also supports alerts for missed doses or low medicine supplies, ensuring reliability and safety.

1.3 Purpose of the Project

The main purpose of this project is to improve medication adherence through automation and voice-based interaction. It reduces dependency on memory, minimises errors, and supports patients who may have difficulty managing medicines manually. For caregivers, it provides an efficient monitoring system that saves time and ensures accuracy. The system ultimately aims to make healthcare more accessible, especially or elderly and disabled users, while offering a scalable solution adaptable to hospitals, clinics, or home environments.

1.4 Scope of the Project

The project's scope covers both patient and caregiver needs.For patients, it enables natural voice communication, automatic dispensing, and real-time feedback.
For caregivers, it allows remote management of schedules and prescriptions through a secure web platform. The design is modular, allowing future integration with healthcare databases, mobile apps, or wearable devices.

Technically, the project demonstrates how AI and IoT can merge to solve real-world healthcare challenges. It shows how intelligent automation can enhance convenience, reliability, and independence for users while maintaining high accuracy in medicine delivery.

1.5 Conclusion

The Intelligent Voice-Driven Smart Medicine Dispenser provides a reliable and user-friendly way to manage medications. By combining NLP for voice understanding and IoT for automated dispensing, the system ensures timely, accurate, and safe medicine delivery. It not only reduces human error but also empowers patients to take charge of their health. This project highlights how technology can bridge the gap between prescribed treatment and real-world adherence, offering a smarter, more compassionate approach to healthcare.

Chapter 2
Literature Survey

2.1 Related works

Paper 1: Gargioni et al. (2024) – Systematic Review on Pill & Medication Dispensers

This paper presents a systematic review of medication dispenser systems from 2000–2023.
It classifies systems based on automation, sensing, connectivity, and interaction methods.
The study shows a shift from basic reminder boxes to IoT-enabled smart dispensers.
Challenges such as non-adherence, elderly usability, and system reliability are highlighted.

Relevance to SmartMeds:

This review justifies the need for an IoT-based smart dispenser like SmartMeds.
Identified challenges directly match SmartMeds' design objectives.
It validates the inclusion of sensors, cloud monitoring, and caregiver access.

Paper 2: Roumaissa (2022) – IoT-Based Pill Management System

The paper proposes an IoT-based pill management system for elderly users.
It integrates a smart dispenser, mobile app, and cloud backend.
Caregivers receive alerts for missed or delayed medication.
Results show improved adherence and user friendliness.

Relevance to SmartMeds:

This architecture closely resembles the SmartMeds system design.
It supports SmartMeds' use of cloud connectivity and caregiver notifications.
The results strengthen confidence in SmartMeds' IoT-based approach.

Paper 3: Pak et al. (2012) – Smart Medication Dispenser with High Reliability

This work introduces a highly reliable smart medication dispenser.
It supports authentication, multi-user management, and remote supervision.

The system emphasizes fault tolerance and error detection.

Clinical evaluations confirm long-term reliability.

Relevance to SmartMeds:

This paper provides core reliability principles for SmartMeds.

It supports the need for error handling and secure dispensing.

SmartMeds adopts similar safety-focused design considerations.

Paper 4: Peddisetti & Panangadan (2024) – IoT Pill Dispenser with Smart Cup

The paper presents an IoT pill dispenser integrated with a smart cup.

Sensors confirm both pill dispensing and actual intake.

Cloud connectivity enables real-time adherence monitoring.

The system overcomes limitations of reminder-only solutions.

Relevance to SmartMeds:

This work supports SmartMeds' sensor-based verification approach.

It highlights the importance of confirming actual medicine consumption.

SmartMeds follows a similar philosophy to improve adherence accuracy.

Paper 5: Zakeri et al. (2022) – ML for Predicting Medication Adherence

This systematic review studies ML models for predicting medication adherence.

Models such as logistic regression, SVM, random forest, and deep learning are analyzed.

Behavioral and historical adherence data are key features.

ML models improve early detection of non-adherence risk.

Relevance to SmartMeds:

This paper supports SmartMeds' future ML-based prediction module.

It validates using historical dispensing and behavior data.

The study guides model selection for SmartMeds' analytics layer.

Paper 6: Mirzadeh et al. (2022) – ML-Based Adherence Prediction

The paper applies ML techniques to real healthcare adherence data.

It focuses on feature engineering and model evaluation.

High prediction accuracy is achieved for non-adherence detection.

Personalized and proactive intervention is emphasized.

Relevance to SmartMeds:

This work provides a practical ML implementation reference.

It aligns with SmartMeds' patient-centric personalization goal.

The pipeline can be adapted for SmartMeds' adherence data.

Paper 7: Trabelsi et al. (2022) – Evaluation of Offline Speech Recognition Toolkits

This paper evaluates offline speech recognition toolkits for embedded systems.

Kaldi and Vosk are assessed for accuracy and resource usage.

Results show acceptable performance on low-power devices.

Offline ASR offers privacy and reliability benefits.

Relevance to SmartMeds:

This study validates SmartMeds' offline voice interaction choice.

It supports deployment on Raspberry Pi–based systems.

Privacy benefits are critical for healthcare applications like SmartMeds.

Paper 8: Lazzaroni et al. (2024) – Embedded Offline Voice Assistant

This paper presents an offline voice assistant for embedded hardware.

Speech recognition, NLU, and response generation run locally.

The system achieves near cloud-level accuracy with low latency.

It is designed for assistive and healthcare use cases.

Relevance to SmartMeds:

This work strongly supports SmartMeds' voice-enabled interface.

It proves offline assistants are feasible for healthcare devices.

SmartMeds benefits from improved accessibility and reliability.

2.2 EXISTING SYSTEMS

Comparative Study

Feature	Mobile Reminder Apps	IoT Pill Boxes	Commercial Automatic Dispensers	Voice-Based Assistants (Alexa/Google)	SmartMeds (Proposed System)
Medication Dispensing	No (reminder only)	No (manual removal)	Yes (automatic)	No	Yes (servo motor–based automated dispensing)
Voice Command Support	No	No	Limited (sensors sometimes)	No	Yes
Verification of Intake	No	Yes (automatic)	Yes (IR/weight sensors)	No	Yes (IR/weight sensors)
Real-Time Cloud Logging	Yes	Yes	Limited	Yes (full web portal)	Yes (full web portal)
Caregiver Dashboard	Yes	No	No	No	Yes
Offline Functionality	No	Medium	High	Low	Low
Hardware Cost	Low	Medium	Medium	Low	Low (Raspberry Pi–based)
Accessibility for Elderly/Visually Impaired	Medium	Low	Medium	High	High (voice + audio prompts)
Customization / Open Source	No	Limited	High	High (customizable)	High (open source + secure APIs)
Target Users	General users	Elderly	Chronic patients	Elderly / visually impaired / caregivers	Elderly / visually impaired / caregivers

HARDWARE AND SOFTWARE REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

Sl. No	Component	Specification / Description
1	Raspberry Pi	Raspberry Pi 4 Model B (4GB RAM)
2	DC Motors	12V DC Motors for medicine dispensing
3	Servo Motor	SG90 Micro Servo
4	Motor Driver	L298N Dual H-Bridge Motor Driver
5	LCD Display	16×2 LCD with I2C Module
6	LEDs	LEDs for status indication
8	Microphone	USB / I2S Microphone for voice input
9	Speaker	USB / 3.5 mm Speaker for voice alerts
10	Power Supply	5V 3A Adapter for Raspberry Pi
11	Connecting Wires	Jumper Wires (Male–Male, Male–Female)
12	Medicine Compartments	Custom 3D-printed / Mechanical Compartments

3.2 SOFTWARE REQUIREMENTS

Sl. No	Software / Tool	Version	Purpose
1	Operating System	Raspberry Pi OS (64-bit, Bookworm)	Raspberry Pi platform
2	Programming Language	Python 3.11	Raspberry Pi control & backend logic
3	Frontend Framework	React.js 18	Web-based patient & caregiver interface
4	Backend Runtime	Node.js 20.x	API & NLP service integration
5	NLP Library	spaCy 3.7	Medicine name & intent extraction
6	Speech-to-Text Library	SpeechRecognition 3.10 / Whisper	Audio-to-text conversion
7	Text-to-Speech Engine	eSpeak-NG 1.51	Voice alerts on Raspberry Pi
8	Database	Firebase Realtime Database	Cloud data storage
9	Cloud Services	Firebase Console	Authentication & database management
10	HTTP Client	Requests (Python) 2.31	Firebase communication
11	GPIO Library	RPi.GPIO 0.7.1	Hardware pin control
12	Motor Control Library	pigpio 1.78	Precise motor & servo control
13	Development Environment	Visual Studio Code	Code development
14	Version Control System	Git 2.40	Source code management

4. SOFTWARE REQUIREMENTS SPECIFICATION

4.1 USERS

The system includes multiple user roles, each with specific tasks and access levels to ensure smooth and secure operation.

4.1.1 Patient

Patients are the primary users of the system. They interact with the dispenser mainly through voice commands recorded via the web interface. Instead of pressing buttons or navigating menus, patients can simply speak naturally — for example, saying “dispense my morning medicine.”

They also receive real-time notifications through audio messages, lights, or on-screen alerts, guiding them through the process and confirming successful dispensing. Additionally, `` patients can view reminders and track their dispensing history, helping them stay consistent with their medication schedule.

4.1.2 Caregiver

Caregivers play a crucial role in managing patients and their medication routines. They can register and manage multiple patients through the web dashboard. For each patient, the caregiver can assign medicines, dosages, and schedules, ensuring accurate prescription management.

Caregivers can also set up reminders and alerts to notify patients about upcoming doses or missed medicines. Through remote monitoring features, they can track adherence and ensure that patients are following their prescribed medication plans without needing to be physically present.

4.1.3 System Administrator

The system administrator oversees the overall functioning of the entire system. Their responsibilities include configuring system settings, maintaining the central database, and managing user access controls to ensure data security and privacy.

Administrators also monitor system health and activity logs to detect performance issues or unusual behaviour. In case of any technical errors, they handle troubleshooting, updates, and maintenance tasks, ensuring the system runs efficiently and reliably for all users.

4.2 Functional Requirements

4.2.1: User Authentication and Authorisation

The system must ensure secure login for all users — including patients, caregivers, and administrators — through cloud-based authentication. Every user must log in with valid credentials before accessing any system feature.

Role-based access control ensures that only authorised users can perform specific actions. For example, patients can request medicines, caregivers can manage schedules, and administrators can configure the system. This prevents unauthorised access and safeguards sensitive medical data.

4.2.2: Patient Profile Management

Caregivers can create, edit, and manage detailed patient profiles within the web application. Each profile stores important details such as the patient's personal information, assigned caregiver, prescribed medicines, and schedules.

Every patient has a unique profile to ensure that all voice commands and dispensing actions are correctly linked. This organised structure also allows caregivers to efficiently manage multiple patients at once.

4.2.3: Medicine Assignment and Scheduling

Caregivers can assign medicines to patients by defining the dosage, dispensing time, and storage compartment for each drug. All this information is stored in the cloud database.
The system supports real-time updates, meaning caregivers can adjust medicine types, doses, or timings remotely if a doctor changes a prescription. This flexibility ensures that patients always receive accurate medication.

4.2.4: Voice Command Recording

Patients can record their requests using a web-based voice interface. This feature removes the need for typing or manual input, making the system more user-friendly for elderly or physically challenged individuals.
The interface provides simple buttons to start or stop recording and clear prompts to ensure the captured voice is of good quality for accurate processing.

4.2.5: Speech-to-Text Conversion

Once a patient records a command, the system automatically converts their speech into text using a speech-to-text engine.
This module ensures accurate transcription even when patients have different accents or speech patterns. The resulting text serves as the basis for further analysis and understanding by the NLP system.

4.2.6: NLP-Based Intent Detection

The converted text is processed using Natural Language Processing (NLP) techniques to determine the user’s intent.
For example, the system identifies whether the patient wants to dispense medicine, check a reminder, or get schedule details. This intelligent understanding allows the system to respond naturally to spoken commands rather than depending on fixed keywords.

4.2.7: Medicine Name Recognition and Matching

The NLP module also extracts medicine names from the patient's command and matches them with the medicines listed in their prescription.
It handles close matches, partial names, and pronunciation differences to minimise errors.
This ensures that even if a patient mispronounces a medicine, the system still identifies it correctly.

4.2.8: Validation of Dispensing Requests

Before dispensing, the system verifies each request to ensure that the medicine is scheduled for the current time, is prescribed to the patient, and has not already been dispensed.
This safety check prevents issues like double dosing or unauthorised dispensing, keeping patients safe and maintaining accurate medicine records.

4.2.9: Automated Medicine Dispensing

Once validation is successful, the system automatically dispenses the correct medicine using Raspberry Pi hardware. The device controls DC motors and servo motors to release medicines from the correct compartments.
This automated mechanism eliminates manual handling and ensures precision and consistency in every dispensing cycle.

4.2.10: Real-Time Cloud Synchronisation

The system continuously synchronises all actions — including medicine schedules, commands, and dispensing results — with the cloud database.
This ensures that both the web application and the hardware unit always share the same data.
Caregivers can monitor patient activity remotely and view real-time updates about medicine intake and dispenser status.

4.2.11: Alerts and Notifications

The system provides clear feedback during every operation. Audio messages, LED indicators, and LCD display notifications inform users about the dispensing status — such as when the process is starting, completed, or if there's an error.

These alerts help patients follow the process easily and reduce confusion during use.

4.2.12: Error Detection and Handling

If a hardware, software, or network problem occurs, the system automatically detects it and notifies the relevant users.

Common issues such as motor failure, internet disconnection, or processing errors trigger alerts and are logged in the cloud database. This transparency helps caregivers or administrators take timely corrective action, ensuring system reliability and patient safety.

4.3 NON-FUNCTIONAL REQUIREMENTS

4.3.1 Performance Requirements

The system must deliver fast and efficient performance with minimal waiting time for all key operations.

When a patient gives a voice command, the system should process it quickly — converting speech to text, analysing intent through NLP, and providing feedback almost instantly. The Raspberry Pi should respond to cloud instructions without noticeable delay, and real-time synchronisation between the web app and cloud database must be maintained. Consistent and quick performance builds user confidence and ensures patients receive their medication on time.

4.3.2 Reliability Requirements

Reliability is essential since errors in dispensing could harm patient health. The system must work consistently and accurately under both normal and challenging conditions.

It should always dispense the correct medicine at the right time, even after prolonged use. If

the network connection is lost, the system must recover automatically when connectivity returns. Hardware components like motors should withstand frequent operations without failure. All failures must be logged so that maintenance staff can review them and maintain dependable long-term performance.

4.3.3 Availability Requirements

The system should remain operational whenever medication dispensing is needed.
Since patients may require medicines at any time, high system availability is critical. The web platform, cloud services, and Raspberry Pi hardware should function with minimal downtime. Routine maintenance must not interrupt essential dispensing schedules, and the system should restart automatically after power outages to maintain continuous service.

4.3.4 Security Requirements

All user data and communication within the system must be protected from unauthorised access.Secure authentication ensures that only verified patients, caregivers, or administrators can log in. Role-based permissions limit who can edit medicine schedules or modify system settings. Data stored in the cloud must follow strict access control policies, and all communication between the web interface, cloud, and hardware should be encrypted to prevent data theft or tampering.

4.3.5 Privacy Requirements

Patient privacy and confidentiality must always be preserved.
The system should collect only the data necessary for operation and avoid storing unnecessary personal or voice information. Sensitive details such as prescriptions and schedules must remain accessible only to authorised users. Following ethical data-handling practices is essential to maintain trust and comply with healthcare standards.

4.3.6 Usability Requirements

The system should be simple and comfortable to use, especially for elderly or non-technical users.The interface must be clear and intuitive, minimising complex steps. Voice commands make it easy for users to interact without typing. Visual indicators on the LCD screen and

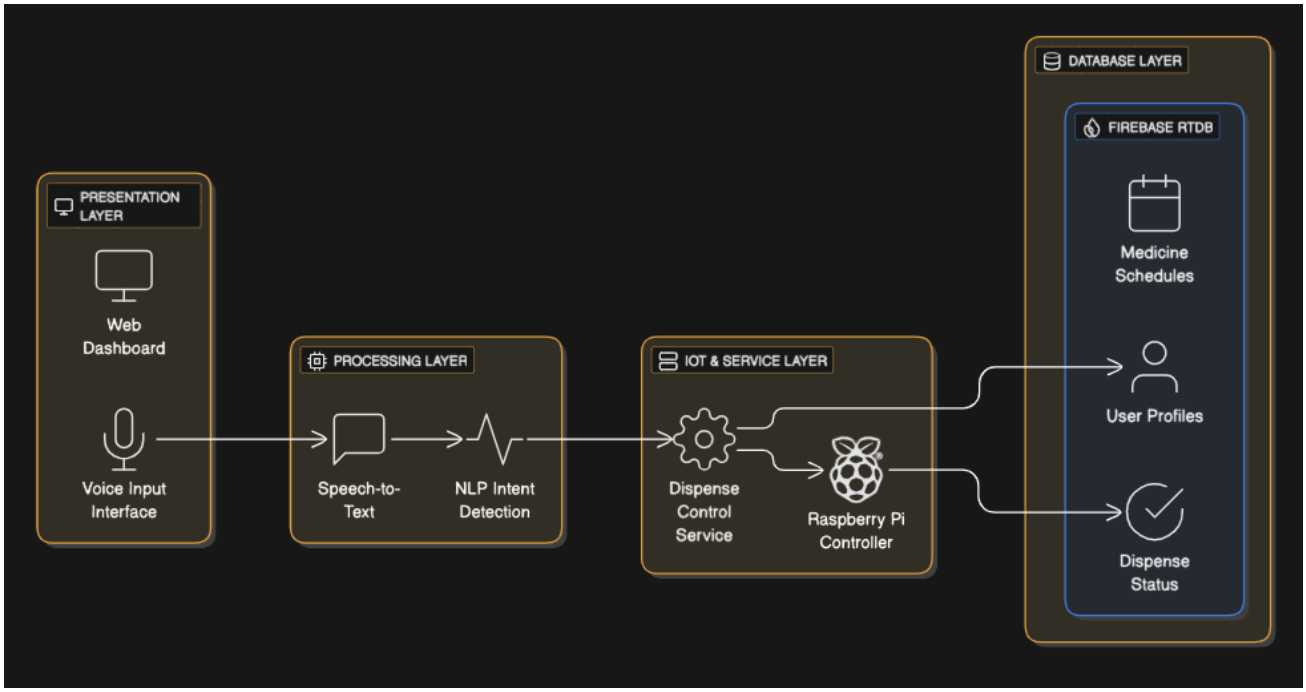


Fig 5.1 Architecture Diagram

LED lights, along with audio guidance, help users understand system status and actions easily. The design should provide smooth navigation and clear feedback for every operation.

4.3.7 Accessibility Requirements

The system must be inclusive and usable by people with physical or sensory impairments. Voice control allows visually impaired users to operate the dispenser, while audio prompts ensure they receive feedback even without reading the display. Simple language, clear messages, and easily distinguishable alerts support users with hearing or cognitive challenges. These features make the system more adaptable to diverse user needs.

4.3.8 Scalability Requirements

The system should support future expansion in both the number of users and added features. Its cloud-based design enables one caregiver to manage multiple patients efficiently without

reducing performance. New modules such as mobile notifications, analytics, or AI-based adherence tracking can be added in the future without major architectural changes, ensuring long-term flexibility and growth.

4.3.9 Maintainability Requirements

The system should be easy to maintain, update, and troubleshoot.
Its modular structure allows developers to update or replace components — such as the web app, NLP service, or hardware control — independently. Clear logging and error reporting simplify debugging. Well-documented code and configuration details ensure smooth maintenance and handover to new developers or administrators.

5. SYSTEM DESIGN

5.1 Architecture Diagram

Presentation Layer

The **Presentation Layer** acts as the primary interaction point between users and the system. It provides user-friendly interfaces through which patients and caregivers can interact with the Smart Medicine Dispenser. This layer consists of two major components: the **Web Dashboard** and the **Voice Input Interface**.

The Web Dashboard is used mainly by caregivers and system administrators to view patient details, monitor medicine schedules, and track dispensing status. It presents information in a clear and structured manner, allowing caregivers to manage medication plans remotely without physical access to the device.

The Voice Input Interface is designed primarily for patients, especially elderly or visually impaired users. Through this interface, patients can issue voice commands such as requesting medicine dispensing. This interface captures the user's voice input and forwards it to the processing layer for further interpretation. By providing voice-based interaction, the system improves accessibility and reduces dependence on manual input.

Processing Layer

The **Processing Layer** is responsible for interpreting user input and converting it into meaningful system commands. This layer bridges the gap between raw user input and system logic. It includes two core components: **Speech-to-Text** and **NLP Intent Detection**.

The Speech-to-Text component converts the captured audio input from the presentation layer into textual form. This step is critical, as accurate transcription ensures reliable downstream processing. Once the speech is converted into text, it is passed to the NLP Intent Detection module.

The NLP Intent Detection component analyses the transcribed text to determine the user's intent. For example, when a user says "dispense paracetamol," this module identifies the intent to dispense medicine and extracts relevant entities such as the medicine name. This intelligent processing allows the system to understand natural language commands rather than relying on rigid input formats.

IoT & Service Layer

The **IoT & Service Layer** serves as the core operational layer of the system, where business logic and physical device control are handled. This layer consists of the **Dispense Control Service** and the **Raspberry Pi Controller**.

The Dispense Control Service acts as the decision-making unit. Based on the intent identified in the processing layer, it validates the request, checks medicine schedules, and determines whether dispensing should proceed. It also communicates with the database to update dispensing status and ensure adherence to medication schedules.

The Raspberry Pi Controller represents the IoT component of the system. It continuously listens for updates from the service layer and the database. Once a valid dispense instruction is received, the Raspberry Pi executes the command by controlling the connected hardware mechanisms. This separation ensures that high-level logic remains independent of low-level hardware operations, improving system flexibility.

Database Layer

The **Database Layer** is responsible for persistent data storage and real-time synchronisation. The system uses **Firebase Realtime Database (RTDB)** to store and manage critical data such as **User Profiles**, **Medicine Schedules**, and **Dispense Status**.

User Profiles store patient and caregiver information, enabling role-based access and personalised medication management. Medicine Schedules define what medicine should be dispensed, in what quantity, and at what time. Dispense Status records the outcome of each dispensing operation, including timestamps, which helps caregivers monitor adherence and detect missed doses.

Firebase RTDB enables real-time data updates, allowing the Raspberry Pi and backend services to stay synchronised without delays. This real-time capability is essential for timely medicine dispensing and accurate status monitoring.

5.2 Process Flow Diagram

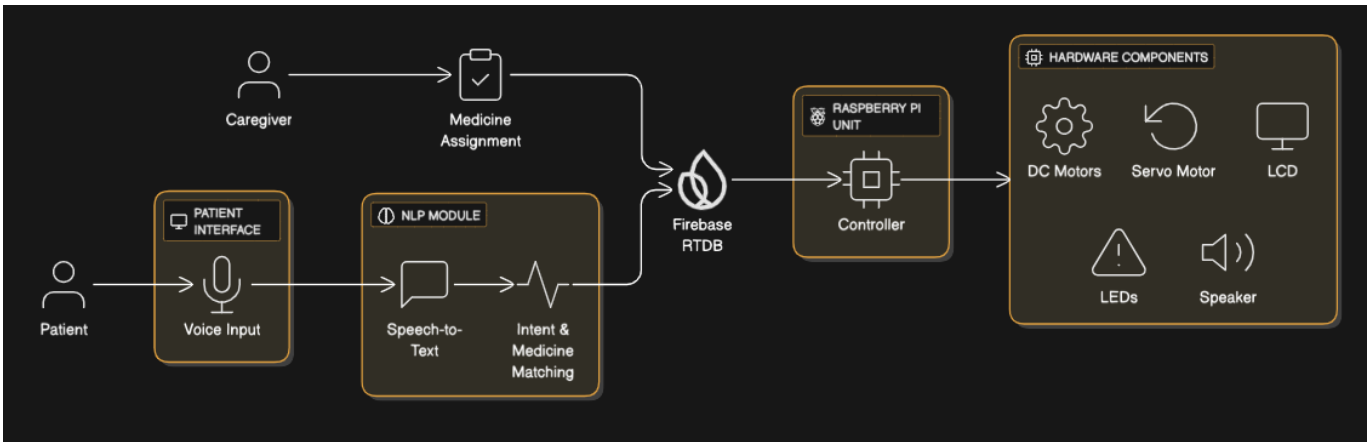


Fig 5.2 Process-Flow Diagram

The **Process Flow Diagram** provides a high-level overview of the **Smart Medicine Dispenser System**, showing how it interacts with external entities such as patients, caregivers, the cloud database, and hardware components. It outlines the system's boundaries and illustrates how data flows between users and system elements without exposing internal technical details.

Patient

The Patient is the primary external user who interacts with the system through a web interface using voice commands. Patients can request medicine dispensing or reminders by speaking naturally. The system captures this audio, converts it to text using NLP (Natural Language Processing), and analyses it to determine the intent and identify the correct medicine.
Once processed, the command is sent to the cloud database for validation and further action.
Patients also receive real-time feedback through audio messages, LED indicators, and on-screen alerts.

Caregiver

The Caregiver manages and supervises patient-related activities. Through the web application, caregivers can register patients, assign medicines, set dosages, and define dispensing times.
All caregiver inputs are stored in the cloud database, ensuring that the latest data is always available

to the system. Although caregivers do not operate the hardware directly, they can monitor patient adherence and track system status remotely in real time.

Cloud Database (Firebase Realtime Database)

The Cloud Database acts as the central communication hub connecting the web application and the Raspberry Pi. It stores all essential information, including patient details, medicine schedules, and system updates.

Both the software and hardware components synchronise with the database in real time. This ensures consistency between caregiver updates, patient commands, and dispenser actions, allowing seamless coordination between all system parts.

Raspberry Pi Unit

The Raspberry Pi serves as the control unit that executes medicine dispensing operations. It continuously monitors the cloud database for dispensing instructions.

When a valid command is received, it activates DC motors, servo motors, and other connected components to release the correct medicines. The unit also controls LEDs, LCD displays, and audio alerts to guide the patient during the process.

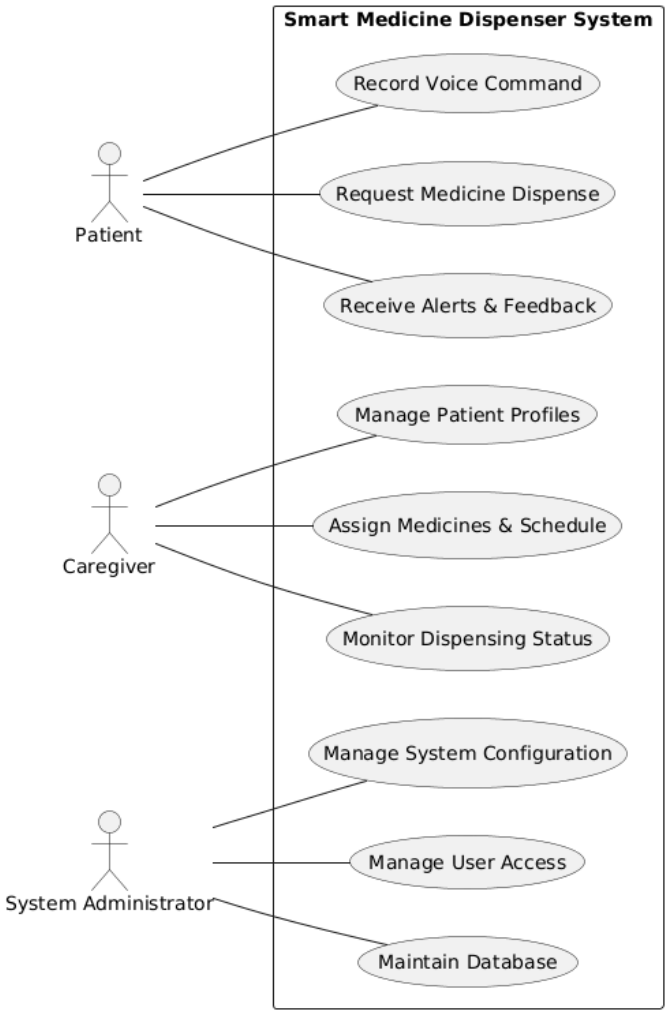
Hardware Components

The Hardware Components form the physical layer of the system. Motors and servos handle medicine dispensing, while LEDs, LCD screens, and speakers provide feedback on the system's status — such as "dispensing," "completed," or "error."

These components ensure accurate dispensing and clear communication with the patient.

6. Detailed Design

6.1 Use-case Diagram



The Use Case Diagram represents how different external users interact with the Smart Medicine Dispenser System. It provides a functional view of the system from the user's perspective, showing the various actions each role can perform and how these actions connect to the system's main processes.

Patient

The Patient is the primary end user and the main beneficiary of the system. Patients interact with the dispenser through a web interface by recording voice commands to request medicine dispensing or to check reminders.

The system processes these commands using Natural Language Processing (NLP) and

provides real-time feedback through audio alerts, LCD messages, and LED indicators. This interaction ensures that patients clearly understand the system's status during medicine intake and receive timely reminders for upcoming doses. The patient's role focuses mainly on issuing commands and receiving feedback in a simple, voice-driven manner.

Caregiver

The Caregiver performs a supervisory and management role within the system. Through the web application, caregivers can create and manage patient profiles, assign medicines, define dosages and schedules, and monitor medicine dispensing remotely.

By using this interface, caregivers can ensure that patients follow their prescribed treatment plans even without being physically present. The caregiver's role strengthens medication adherence and allows for centralised management of multiple patients.

System Administrator

The System Administrator manages the technical and operational side of the Smart Medicine Dispenser. This role includes configuring the system, controlling user access permissions, and maintaining the cloud database.

Administrators also handle software updates, troubleshoot errors, and ensure the system remains reliable, secure, and available. Their actions maintain data integrity and ensure that all user interactions occur smoothly within the system environment.

Overall Use Case Interaction

Each use case in the diagram represents a unique function or operation, such as "Record Voice Command," "Dispense Medicine," "Manage Patient Profile," or "Update Configuration."

The diagram visually separates the roles of Patient, Caregiver, and System Administrator while showing how their interactions connect within the same system boundary. By organising these actors clearly, the diagram improves readability and effectively communicates the role-based relationships between users and system functionalities.

6.2 Activity Diagram

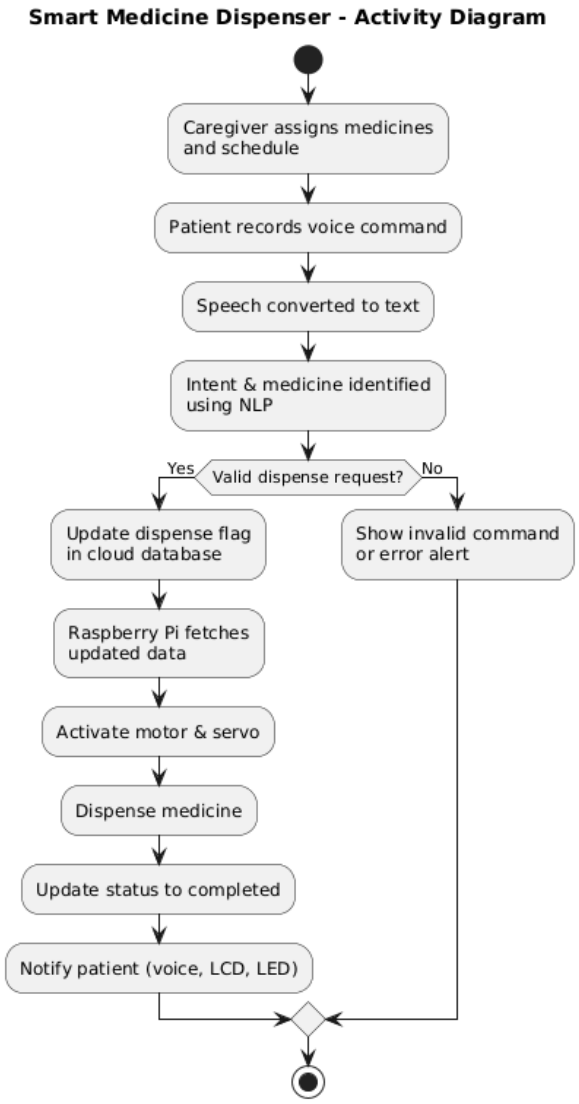


Fig 6.2 Activity Diagram

The activity diagram illustrates the simplified operational flow of the Smart Medicine Dispenser system from configuration to medicine dispensing. The process begins with the caregiver assigning medicines and dosage schedules through the web application. These details are stored in the cloud database, making them available to the system in real time.

The patient then interacts with the system by recording a voice command through the web interface. The recorded audio is converted into text and processed using Natural Language Processing techniques to identify the user's intent and match medicine names

with the prescribed list. If the system determines that the request is valid, a dispense flag is updated in the cloud database.

The Raspberry Pi continuously monitors the cloud database for dispensing instructions. Upon detecting a valid dispense request, it activates the connected DC motors and servo motors to release the medicine from the appropriate compartment. After successful dispensing, the system updates the dispensing status in the cloud and provides feedback to the patient through audio alerts, LCD messages, and LED indicators. If the request is invalid, an alert is generated and shown to the patient.

This simplified activity diagram provides a clear overview of the end-to-end workflow and highlights the coordination between the web application, NLP processing, cloud services, and embedded hardware unit.

6.3 DATABASE DESIGN

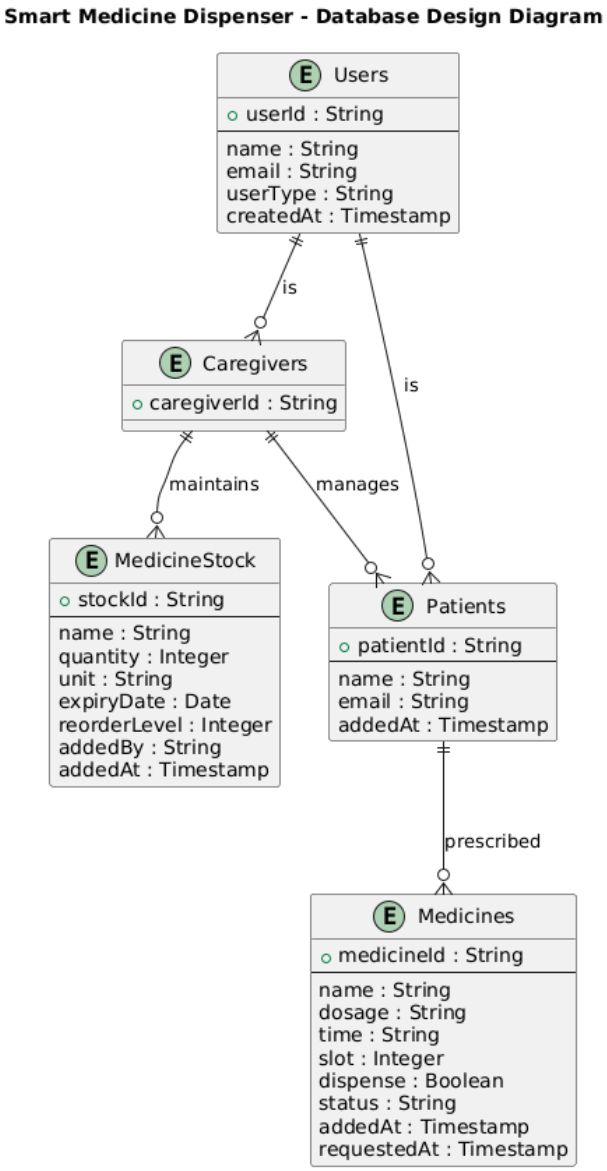


Fig 6.3 Database Design

The database design follows a role-based and patient-centric structure, optimized for real-time synchronization between the web application and the Raspberry Pi dispensing unit.

The Users entity stores common authentication and identity details for all system users. Each user is classified by a userType, which determines whether the user functions as a patient or a caregiver. This avoids duplication and simplifies authentication logic.

The Caregivers entity represents users responsible for managing patients and medicines. Each caregiver can manage multiple patients, establishing a one-to-many relationship. This design supports real-world caregiving scenarios where a single caregiver supervises multiple individuals.

The Patients entity stores patient-specific details and is linked to caregivers. Each patient has a unique identifier, ensuring that voice commands and dispensing actions are mapped correctly to the respective patient.

The Medicines entity is associated with patients and stores prescription details such as medicine name, dosage, time of intake, compartment slot, and dispensing status. The dispense flag and status field are critical for triggering and tracking automated dispensing operations by the Raspberry Pi.

The MedicineStock entity maintains inventory information managed by caregivers. It tracks available quantities, expiry dates, and reorder levels, enabling future extensions such as low-stock alerts and inventory management.

7 IMPLEMENTATION

7.1 Pseudo Code

7.1.1 Frontend (React Web Application)

```
FUNCTION App():
   STATE authView = 'loading'  // 'loading', 'login', 'signup', 'home'
   STATE user = null
   STATE userType = null  // 'patient' or 'caregiver'

   ON_MOUNT:
      auth = initializeFirebaseAuth()

      LISTEN_TO auth.onAuthStateChanged:
         IF user_authenticated:
            user = getCurrentUser()
            userType = FETCH_FROM_RTDB('users/{uid}/userType')
            authView = 'home'
         ELSE:
            authView = 'login'

   FUNCTION handleLogout():
      CALL signOut(auth)
      user = null
      userType = null
      authView = 'login'

   RENDER:
      IF authView == 'loading':
         SHOW LoadingSpinner

      ELSE IF authView == 'login':
         SHOW Login(onSuccess = handleAuthSuccess)

      ELSE IF authView == 'signup':
         SHOW Signup(onSuccess = handleAuthSuccess)

      ELSE IF authView == 'home':
         IF userType == 'patient':
            SHOW PatientDashboard(user, onLogout)
         ELSE IF userType == 'caregiver':
            SHOW CaregiverDashboard(user, onLogout)
```

```
FUNCTION AudioRecorder(user, onDetection):
  STATE recording = null
  STATE isRecording = false
  STATE audioBlob = null
  STATE transcript = ""
  STATE processing = false

  FUNCTION startRecording():
    stream = getUserMedia({audio: true})
    mediaRecorder = NEW MediaRecorder(stream)
    mediaRecorder.start()
    isRecording = true

  FUNCTION stopRecording():
    recording.stop()
    processAndUpload(audioBlob)

  FUNCTION processAndUpload(audioBlob):
    formData = NEW FormData()
    formData.append('audio', audioBlob)

    response = FETCH(BACKEND + '/process_audio?patient_uid=' + user.uid, {
      method: 'POST',
      body: formData
    })

    data = AWAIT response.json()
    transcript = data.text

    IF data.medicine_name:
      onDetection({
```

```
            medicine_name: data.medicine_name,
            dosage: data.dosage,
            slot: data.slot
        })
FUNCTION MedicineDisplay(user):
  STATE medicines = []

  ON_MOUNT:
    medicinesRef = ref(rtdb, 'medicines/{user.uid}')
    LISTEN_TO medicinesRef:
      medicines = CONVERT_TO_ARRAY(data)

  FUNCTION sendInstruction(medicineName, dosage):
    response = FETCH(BACKEND + '/send_instruction', {
      method: 'POST',
      body: JSON.stringify({
        medicine_name: medicineName,
        patient_uid: user.uid,
        dosage: dosage
      })
    })

    IF response.ok:
      SHOW_SUCCESS("Medicine ready to dispense")
```

7.1.2 Backend(Flask API Server)

```
ENDPOINT POST /process_audio:
  audioFile = request.files['audio']
  patient_uid = request.args.get('patient_uid')
```

```
    // Convert audio to WAV
    wavPath = CONVERT_TO_WAV(audioFile)

    // Speech recognition
    recognizer = SpeechRecognizer()
    text = recognizer.recognize_google(wavPath)

    // Extract medicine name using NLP
    medicineName = EXTRACT_MEDICINE_NAME(text)
    dosage = EXTRACT_DOSAGE(text)

    // Fetch from Firebase
    IF patient_uid:
        medicineDetails = FETCH_FROM_DB(patient_uid, medicineName)
        RETURN {
            text: text,
            medicine_name: medicineName,
            dosage: dosage,
            slot: medicineDetails.slot,
            status: medicineDetails.status
        }
ENDPOINT POST /send_instruction:
    medicine_name = request.json['medicine_name']
    patient_uid = request.json['patient_uid']

    // Find medicine in Firebase
    medicines = FIREBASE_RTDB.get('medicines/{patient_uid}')

    FOR EACH medicine IN medicines:
        IF medicine.name == medicine_name:
            // Update status to ready_to_dispense
```

```
FIREBASE_RTDB.update('medicines/{patient_uid}/{medicine_id}', {
    status: 'ready_to_dispense',
    requestedAt: CURRENT_TIMESTAMP
})

RETURN {
    status: 'success',
    slot: medicine.slot
}
```

7.1.3 RaspberryPi

BEGIN

Load environment variables from the .env file

Read FIREBASE_URL and USER_ID

If FIREBASE_URL or USER_ID is not available then

Display configuration error and terminate program

End If

Initialise Firebase communication module

Initialise speaker system for voice alerts

Initialise GPIO pins for DC motors, servo motor, and LEDs

Set GPIO warnings off and configure pin modes

Speak “Smart medicine dispenser is online”

Display system start message

While true do

Fetch medicine data from Firebase using the path:

FIREBASE_URL / medicines / USER_ID

If data fetch fails or returns permission error then

Display error message

Wait for a short interval

Continue to next loop iteration

End If

If fetched medicine data is empty then

Wait for a short interval

Continue to next loop iteration

End If

For each medicine entry in the fetched data do

If the medicine entry is not a valid object then

Skip this entry

End If

Read dispense flag and status value from the medicine entry

If dispense flag is true (boolean or string)

and status is equal to “pending” then

Read medicine name, dosage, and slot number

Speak “Dispensing <medicine name> with dosage <dosage>”

Start LED loading animation

Activate the DC motor corresponding to the slot number

Rotate motor for a predefined duration

Open the servo motor to release the medicine

Close the servo motor and return to default position

If dispensing operation is successful then

Speak “<medicine name> dispensed successfully”

Turn on green LED indicator

Update Firebase:

Set dispense flag to false

Set status to “dispensed”

Else

Speak “Error dispensing <medicine name>”

Blink red LED indicator

End If

End If

End For

Wait for a predefined interval before the next Firebase check

End While

END

7.2 Screenshots

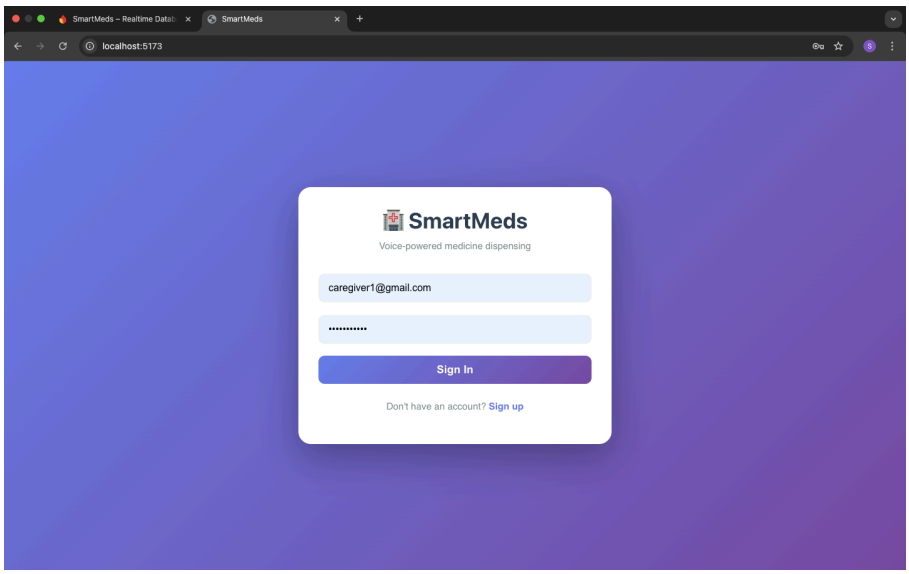


Fig 7.1 Login Page

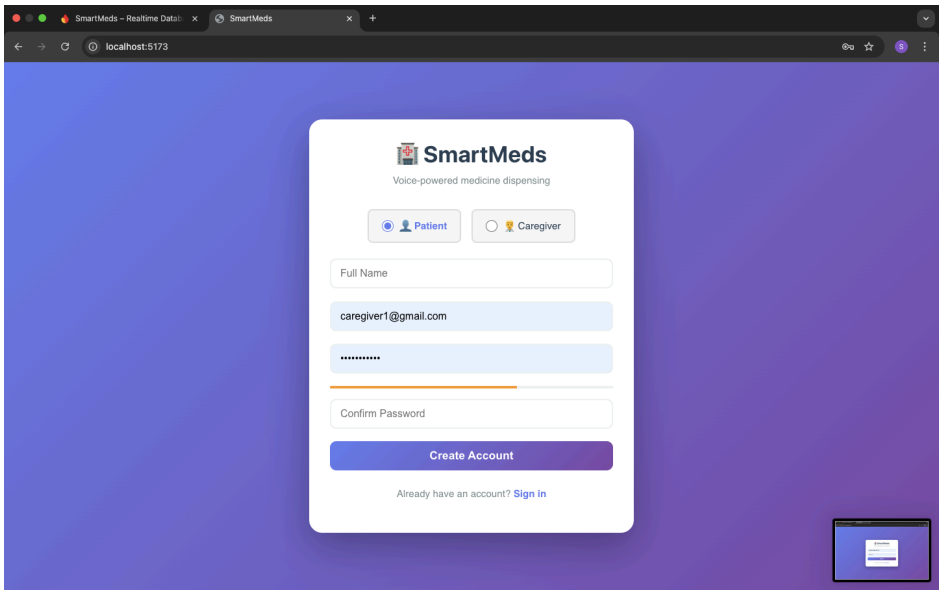


Fig 7.2 Signup Page

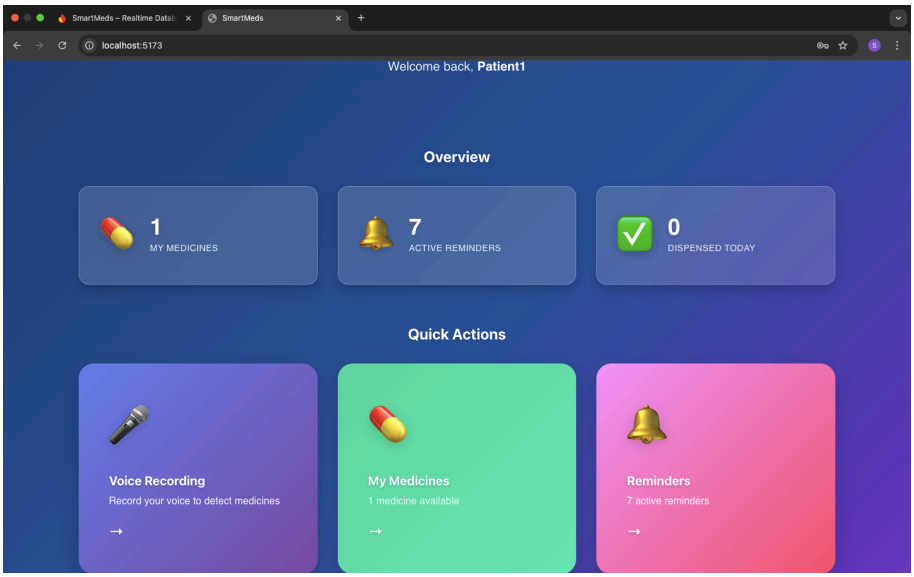


Fig 7.3 Patient Home Page

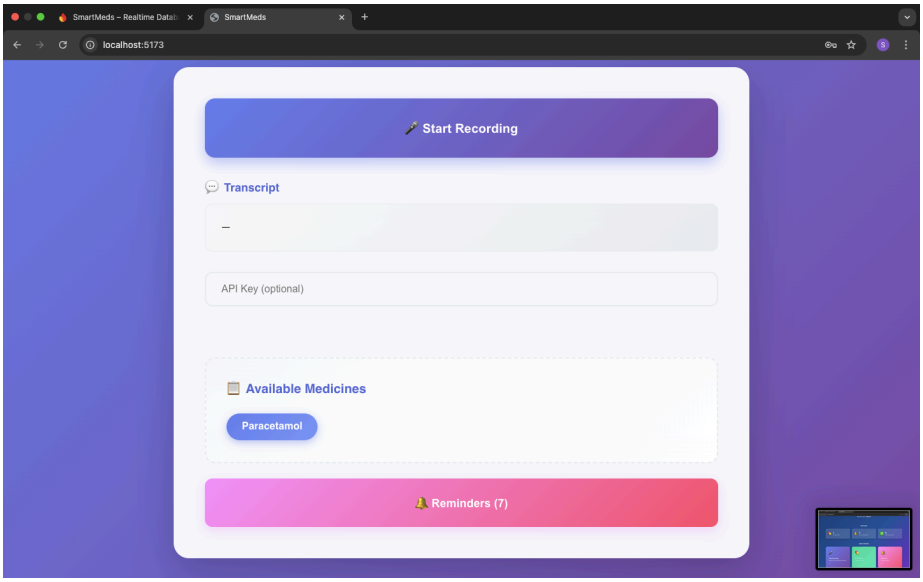


Fig 7.4 Audio Recording Dashboard

- Patient Home (stats, quick actions, recent activity)
- Voice Recording & Medicine Detection (complete workflow)
- My Medicines (medicine list with slot/time/status)
- Reminders (local reminder management)

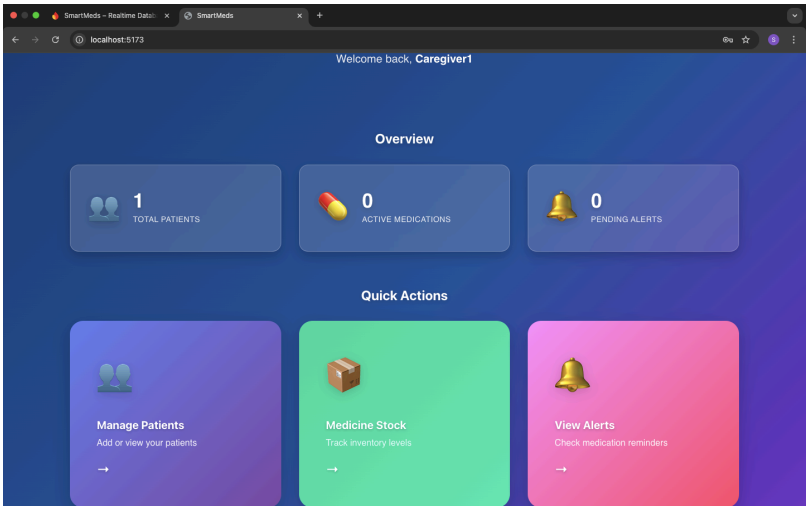
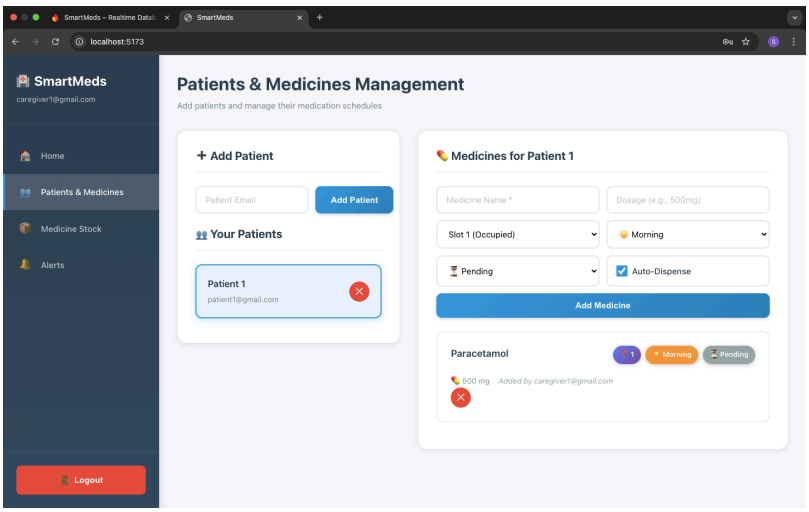


Fig 7.5 Caregiver Dashboard



7.6 Caregiver Dashboard

- Caregiver Home (overview, statistics, quick actions)
- Patient Management (add/remove patients)
- Medicine Manager (slot assignment, medicine details)
- Medicine Stock (inventory tracking, expiry warnings)
- Alerts (priority system, filtering, actions)

Test Cases

Frontend Testcases

Test Case ID	Test Scenario	Input / Action	Expected Output	Actual Result	Status
FE-01	User Login	Enter valid credentials	User successfully logged in and redirected to dashboard	User authenticated successfully and redirected to the appropriate dashboard	Pass
FE-02	Invalid Login	Enter wrong email/ password	Error message displayed	Error message displayed indicating invalid credentials	Pass
FE-03	Role-Based Access	Login as patient	Patient dashboard is displayed	Patient-specific dashboard loaded with relevant features	Pass
FE-04	Role-Based Access	Login as caregiver	Caregiver dashboard is displayed	Caregiver dashboard displayed with medicine and patient management options	Pass
FE-05	Voice Recording Start	Click “Start Recording”	Microphone starts recording	Microphone permission granted and audio recording started	Pass
FE-06	Voice Recording Stop	Click “Stop Recording”	Audio recording stops and uploads	Recording stopped successfully and audio file uploaded	Pass
FE-07	Audio Upload	Submit recorded audio	Backend receives audio successfully	Audio file successfully sent to backend API	Pass
FE-08	Medicine List Fetch	Load dashboard	Medicines fetched from Firebase and displayed	Medicine data fetched from Firebase and displayed correctly	Pass
FE-09	Send Dispense Request	Click dispense button	Success message shown	Dispense request sent and confirmation message displayed	Pass
FE-10	Logout	Click logout	User logged out and redirected to login	Session terminated and user redirected to login page	Pass

Backend Testcases

Test Case ID	Test Scenario	Input / API Call	Expected Output	Actual Result	Status
BE-01	Process Audio API	POST `/process_audio` with valid audio + `patient_uid`	Returns transcript and medicine details	API successfully processed audio and returned transcript along with extracted medicine details	Pass
BE-02	Invalid Audio Input	Corrupted or missing audio file	Error response returned	System returned validation error indicating invalid or missing audio input	Pass
BE-03	Speech-to-Text	Valid voice command	Correct text transcription	Voice command was accurately converted into text	Pass
BE-04	NLP Intent Detection	"Dispense paracetamol"	Medicine name and dosage extracted	NLP module correctly identified intent and extracted medicine name	Pass
BE-05	Medicine Match	Spoken medicine exists in DB	Correct slot and status returned	Medicine matched successfully with database entry and correct slot was identified	Pass
BE-06	Medicine Not Found	Unknown medicine name	Error or null response	System returned appropriate error indicating medicine not found	Pass
BE-07	Send Instruction API	POST `/send_instruction`	Firebase updated to `ready_to_dispense`	Firebase Realtime Database updated successfully with dispense flag	Pass
BE-08	Invalid Patient UID	Wrong `patient_uid`	Error response	API returned authentication/ authorization error	Pass
BE-09	Firebase Update	Valid medicine request	Status updated with timestamp	Medicine status updated correctly with current timestamp in Firebase	Pass
BE-10	API Response Time	Normal load	Response within acceptable time	API responded within acceptable latency limits	Pass

BIBILOGRAPHY

[1] Raspberry Pi Foundation, *Raspberry Pi Documentation*, Raspberry Pi Ltd. [Online]. Available: https://www.raspberrypi.com/documentation/.

[2] Python Software Foundation, *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/.

[3] Google, *Firebase Realtime Database Documentation*. [Online]. Available: https://firebase.google.com/docs/database.

[4] Google, *Firebase Realtime Database Security Rules*. [Online]. Available: https://firebase.google.com/docs/database/security. Accessed: 27-Dec-2025.

[5] K. Reitz et al., *Requests: HTTP for Humans – Python Library*. [Online]. Available: https://docs.python-requests.org/.

[6] S. Frazier, *python-dotenv Documentation*. [Online]. Available: https://pypi.org/project/python-dotenv/.

[7] Raspberry Pi Foundation, *RPi.GPIO Python Library Documentation*. [Online]. Available: https://sourceforge.net/p/raspberry-gpio-python/wiki/Home/.

[8] eSpeak NG Team, *eSpeak NG: Open Source Text-to-Speech Engine*. [Online]. Available: https://github.com/espeak-ng/espeak-ng.

[9] ALSA Project, *Advanced Linux Sound Architecture (ALSA)*. [Online].

[10] GeeksforGeeks Web Development– *Overview*. [Online]. Available: https://www.geeksforgeeks.org/web-tech/web-technology/

[11] TutorialsPoint, *Embedded Systems Tutorial*. [Online]. Available: https://www.tutorialspoint.com/embedded_systems/index.htm.

Research Papers

[1] L. Gargioni, D. Fogli, and P. Baroni, “A systematic review on pill and medication dispensers from a human-centered perspective,” J. Healthcare Informatics Research, Feb. 2024. Available: https://pubmed.ncbi.nlm.nih.gov/38681758/

[2] V. Peddisetti et al., “Smart Medication Management: Enhancing medication adherence with an IoT-based pill dispenser and smart cup,” presented at the IEEE AIMHC 2024, Full PDF available: https://www.fullerton.edu/ecs/faculty/apanangadan/publications/SmartMedicationManagement_AIMHC-2024.pdf

[3] B. Roumaissa, “An IoT-Based Pill Management System for Elderly People,” Informatica, 2022. Available: https://www.informatica.si/index.php/informatica/article/view/4195

[4] S. Faisal et al., “Key features of smart medication adherence products,” Aging and Medication Technologies J., 2023. Available: https://pubmed.ncbi.nlm.nih.gov/36469382/

[5] M. Zakeri et al., “Machine learning in predicting medication adherence: A systematic review,” J. Medical Artificial Intelligence, 2022. Available: https://jmai.amegroups.org/article/view/6666/html

THANK YOU